

ANO
2026



UNINTER

ATIVIDADE PRÁTICA

ARQUITETURA E DESENVOLVIMENTO DE APIS

Prof. Pedro Henrique Flores

Prof. Osmar Dias Jr.

INTRODUÇÃO

Olá a todos.

Sejam todos muito bem-vindos!

Esta avaliação foi planejada e preparada para as disciplinas de Arquitetura e Desenvolvimento de APIs do Centro Universitário Internacional Uninter.

O objetivo desta atividade é fazer com que você, aluno, desenvolva os conhecimentos teóricos e práticos aprendidos na rota de maneira.

Ao longo desse roteiro serão passadas as orientações gerais para realização da avaliação bem como os seus critérios de correção.

No mais, desejo-lhe boa atividade prática em nome dos professores.

SUMÁRIO

ORIENTAÇÕES GERAIS.....	1
OBJETIVO DO TRABALHO.....	1
A PROPOSTA.....	1
API PRÓPRIA X CONSUMO DE API EXTERNA.....	2
REQUISITOS OBRIGATÓRIOS DA APLICAÇÃO	2
OPERAÇÕES MÍNIMAS E ENDPOINTS	3
QUALIDADE DAS RESPOSTAS HTTP.....	4
ORM, CORS E AUTENTICAÇÃO	4
ORM	4
CORS	4
Autenticação e controle de acesso	5
INTEGRAÇÃO OPCIONAL COM APIS PÚBLICAS	5
ENTREGÁVEIS.....	5
REQUISITOS DO README	6
ESTRUTURA SUGERIDA PARA O RELATÓRIO TÉCNICO	6
TABELA DE ENDPOINTS OBRIGATÓRIA NO RELATÓRIO	7
CRITÉRIOS DE AVALIAÇÃO - 100 PONTOS	8
CONDIÇÕES IMPORTANTES PARA A AVALIAÇÃO	9
EXEMPLOS DE TEMAS POSSÍVEIS	9

ORIENTAÇÕES GERAIS

Você pode realizar esta atividade individualmente, em duplas ou trios. Sendo de sua responsabilidade e de seus colegas, o contato com os demais membros da equipe. Utilizem canais de comunicação já familiares para localizar colegas para a equipe.

Caso você esteja realizando o trabalho em equipe, escolha somente um dos integrantes do grupo para postar o relatório no AVA, que deverá ter em mãos os **RUs** e nomes dos demais colegas para informa-los quando estiver postando a versão final.

OBJETIVO DO TRABALHO

O trabalho final tem como objetivo verificar se o estudante é capaz de planejar, desenvolver, documentar e demonstrar uma API própria, utilizando padrões e boas práticas trabalhados ao longo da disciplina.

A avaliação prioriza a compreensão dos conceitos de APIs, e não o domínio de uma linguagem específica. Por isso, o estudante terá liberdade para escolher a linguagem, o framework e o banco de dados que considerar mais adequados, desde que consiga justificar tecnicamente essas escolhas e atender aos requisitos mínimos deste documento.

A PROPOSTA

O estudante deverá desenvolver uma API Web própria, com tema livre, capaz de disponibilizar e manipular recursos por meio de requisições HTTP.

- O domínio da aplicação é **livre**: biblioteca, tarefas, filmes, eventos, produtos, esportes, receitas, finanças pessoais, catálogo, agenda ou outro tema de interesse.
- A linguagem de programação é de **livre escolha**.
- O framework é de **livre escolha**, desde que adequado ao desenvolvimento de APIs.
- O banco de dados também é de **livre escolha**, desde que exista persistência real dos dados.
- A solução deverá ser disponibilizada em um repositório remoto acessível ao professor para avaliação.

Em caso de dificuldade na elaboração ou publicação do trabalho, entre em contato com o tutor da disciplina.

API PRÓPRIA X CONSUMO DE API EXTERNA

A entrega obrigatória é uma API desenvolvida pelo próprio estudante. O simples consumo de uma API pública não caracteriza o desenvolvimento exigido neste trabalho.

O consumo de APIs externas é opcional e pode ser utilizado para enriquecer a solução. Nessa situação, a API criada pelo estudante poderá consultar dados externos, processá-los, combiná-los com informações próprias e disponibilizar novos endpoints.

Situação	Aceita?	Exemplo
Criar uma API própria com banco de dados e CRUD	Sim	API de tarefas, biblioteca, catálogo, agenda etc.
Criar uma API própria que também consome uma API pública	Sim	API de viagens que usa dados da REST Countries, mas possui seus próprios recursos e persistência.
Entregar apenas um programa que consulta uma API pública	Não	Aplicação que apenas busca e exibe dados da PokeAPI sem desenvolver endpoints próprios.

REQUISITOS OBRIGATÓRIOS DA APLICAÇÃO

A API deverá contemplar, no mínimo, os seguintes elementos:

- **API Web baseada em HTTP:** A aplicação deverá receber requisições e devolver respostas HTTP por meio de endpoints próprios.
- **Organização REST/RESTful:** As rotas deverão representar recursos de forma coerente, utilizando adequadamente métodos HTTP, URLs e códigos de resposta.

- **Persistência em banco de dados:** Os dados deverão permanecer armazenados entre diferentes execuções da aplicação. Estruturas temporárias em memória não são suficientes como solução final.
- **ORM:** A aplicação deverá utilizar uma técnica ou ferramenta de mapeamento objeto-relacional compatível com a tecnologia escolhida. A escolha deverá ser explicada no relatório.
- **CRUD mínimo:** A solução deverá demonstrar consulta, criação, atualização e exclusão de recursos.
- **Validação de dados:** Entradas inválidas deverão ser identificadas e tratadas de forma adequada.
- **Tratamento de erros:** A API deverá retornar respostas coerentes em situações como recurso inexistente, dados inválidos ou acesso não autorizado.
- **Códigos HTTP adequados:** Os códigos de status deverão representar corretamente o resultado das operações.
- **Autenticação:** A aplicação deverá apresentar ao menos um mecanismo simples de autenticação ou controle de acesso e demonstrar ao menos um endpoint protegido.
- **CORS:** A política adotada deverá ser configurada e explicada. Caso a aplicação não possua um cliente de outra origem, o estudante deverá explicar a finalidade do CORS e justificar a configuração escolhida para o cenário.
- **Testes e evidências:** Os endpoints deverão ser demonstrados com Swagger/OpenAPI, Postman, Insomnia, cURL, testes automatizados ou ferramenta equivalente.
- **Documentação de execução:** O repositório deverá conter instruções suficientes para instalação, configuração e execução da API.

OPERAÇÕES MÍNIMAS E ENDPOINTS

A API deverá implementar as operações abaixo sobre pelo menos um recurso principal. Os nomes das rotas são definidos pelo estudante conforme o domínio escolhido.

Método	Operação esperada	Pontuação no critério CRUD
GET	Listar uma coleção de recursos.	5 pontos
GET	Consultar um recurso individual e/ou realizar filtragem significativa.	5 pontos

Método	Operação esperada	Pontuação no critério CRUD
POST	Criar um novo recurso.	5 pontos
PATCH ou PUT	Atualizar um recurso existente e justificar a escolha do método.	5 pontos
DELETE	Remover um recurso existente.	5 pontos

Observação: não é obrigatório implementar PATCH e PUT simultaneamente. O estudante deverá escolher pelo menos um deles e justificar sua utilização.

QUALIDADE DAS RESPOSTAS HTTP

Além de executar as operações, a API deverá apresentar respostas coerentes com o resultado de cada requisição. Espera-se o uso adequado de códigos HTTP como 200, 201, 204, 400, 401/403 e 404, conforme o contexto da implementação.

As respostas deverão utilizar uma representação adequada de dados, preferencialmente JSON quando compatível com o framework e o domínio escolhido, e apresentar mensagens de erro compreensíveis quando necessário.

ORM, CORS E AUTENTICAÇÃO

ORM

O relatório deverá identificar a ferramenta ou técnica de ORM utilizada e explicar como classes/modelos da aplicação são relacionadas às estruturas persistidas no banco de dados. Não basta apenas citar o nome da biblioteca.

CORS

O estudante deverá apresentar a política de CORS configurada na aplicação, indicando, quando aplicável, origens, métodos e cabeçalhos permitidos. Configurações

amplas, como a liberação irrestrita de qualquer origem, deverão ser tecnicamente justificadas.

AUTENTICAÇÃO E CONTROLE DE ACESSO

A aplicação deverá implementar uma solução compatível com o nível do projeto, como API Key, Basic Authentication, token, JWT, sessão ou mecanismo equivalente. Deve existir ao menos uma evidência de endpoint protegido e uma breve explicação do fluxo de autenticação.

INTEGRAÇÃO OPCIONAL COM APIS PÚBLICAS

O estudante poderá utilizar uma API pública como fonte de dados ou serviço complementar. Essa integração é opcional e não substitui a API própria.

Sugestões de serviços que podem ser explorados:

API	Possibilidades	Endereço
REST Countries	Dados de países, idiomas, moedas, bandeiras e informações geográficas.	https://restcountries.com
PokeAPI	Dados sobre Pokémon, habilidades, tipos e outros recursos do universo da franquia.	https://pokeapi.co
Open Library	Dados bibliográficos de livros, autores e obras.	https://openlibrary.org/developers/api

ENTREGÁVEIS

A entrega deverá conter os seguintes itens:

- Link do repositório remoto contendo o código-fonte completo da API.
- Arquivo README com instruções para instalação, configuração e execução do projeto.
- Relatório técnico em formato definido pelo professor.
- Arquivo de exemplo para variáveis de ambiente, quando necessário, sem expor senhas, tokens ou chaves reais.
- Evidências de funcionamento das operações e dos principais requisitos.

- Banco, script de criação, migration ou instrução equivalente que permita reproduzir a estrutura de persistência.

REQUISITOS DO README

O README deverá permitir que outra pessoa consiga compreender e executar o projeto sem depender de explicações adicionais do autor. Recomenda-se incluir:

- Descrição da API e do problema resolvido.
- Tecnologias utilizadas.
- Pré-requisitos.
- Instalação das dependências.
- Configuração de banco de dados e variáveis de ambiente.
- Comando ou procedimento para iniciar a aplicação.
- Endereço da documentação/Swagger, quando disponível.
- Resumo dos principais endpoints.
- Exemplo de requisição.
- Informações necessárias para testar autenticação sem expor credenciais sensíveis.

ESTRUTURA SUGERIDA PARA O RELATÓRIO TÉCNICO

O relatório deverá registrar as decisões de projeto e apresentar evidências do desenvolvimento. A estrutura abaixo pode ser utilizada como referência:

Seção	Conteúdo esperado
1. Introdução	Contextualização do problema, objetivo da solução e público ou cenário de uso.
2. Tecnologias utilizadas	Linguagem, framework, banco, ORM, ferramentas de teste e demais recursos.
3. Justificativa das escolhas	Razões técnicas para as principais decisões adotadas.
4. Arquitetura da API	Recursos, endpoints, organização do código e fluxo geral da aplicação.

Seção	Conteúdo esperado
5. Banco de dados e ORM	Entidades/modelos, relacionamentos quando existentes e funcionamento da persistência.
6. Endpoints e HTTP	Métodos, rotas, parâmetros, corpo das requisições, respostas e códigos HTTP.
7. Validação e erros	Estratégias adotadas para validar dados e responder a situações inválidas.
8. Autenticação e CORS	Mecanismo de controle de acesso e política de comunicação entre origens.
9. Testes e evidências	Demonstrações das operações por ferramenta de teste ou testes automatizados.
10. Execução do projeto	Procedimentos necessários para reproduzir o ambiente e iniciar a API.
11. Repositório	Link para o repositório remoto.
12. Considerações finais	Dificuldades encontradas, decisões tomadas, limitações e possíveis melhorias.

TABELA DE ENDPOINTS OBRIGATÓRIA NO RELATÓRIO

O relatório deverá apresentar uma tabela equivalente ao modelo abaixo, preenchida com os endpoints reais da aplicação:

Método	Endpoint	Finalidade	Entrada principal	Respostas esperadas
GET	/recursos	Listar recursos	Filtros opcionais	200
GET	/recursos/{id}	Consultar recurso	Identificador	200 / 404
POST	/recursos	Criar recurso	JSON com dados	201 / 400
PATCH/P UT	/recursos/{id}	Atualizar recurso	Identificador + dados	200 / 400 / 404
DELETE	/recursos/{id}	Excluir recurso	Identificador	204 / 404

CRITÉRIOS DE AVALIAÇÃO - 100 PONTOS

A avaliação será realizada pela combinação entre funcionamento da aplicação, coerência técnica, documentação e capacidade de demonstrar os conceitos da disciplina.

Critério	Pontos	O que será observado
1. Proposta e arquitetura da solução	5	Problema bem definido; recursos coerentes; organização geral da API; adequação da proposta ao escopo da disciplina.
2. Escolha tecnológica e justificativas	10	Linguagem, framework, banco, ORM e ferramentas escolhidos de forma coerente, com justificativas técnicas.
3. Endpoints e operações CRUD	25	GET (listagem), GET (individual/filtro), POST, PATCH/PUT e DELETE funcionando corretamente. 5 pontos por operação.
4. Banco de dados e ORM	15	Persistência funcional, modelagem adequada, mapeamento por ORM e manipulação consistente dos dados.
5. HTTP, validação e tratamento de erros	10	Uso coerente de códigos HTTP, validação de entrada, respostas de erro e tratamento de situações inválidas.
6. Segurança, autenticação e CORS	15	Mecanismo de autenticação/controle de acesso, endpoint protegido e política de CORS tecnicamente explicada.
7. Testes e evidências de funcionamento	10	Demonstração verificável dos endpoints e requisitos por ferramenta de testes ou testes automatizados.
8. Repositório e documentação técnica	10	Repositório organizado, README completo, instruções de execução, relatório e informações suficientes para reprodução do projeto.
Total		100

CONDIÇÕES IMPORTANTES PARA A AVALIAÇÃO

- Uma aplicação que apenas consome uma API externa, sem disponibilizar endpoints próprios, não atende ao escopo principal do trabalho.
- Frontend é opcional. A qualidade visual de uma interface não substitui requisitos da API e não gera pontuação adicional por si só.
- Quantidade de endpoints não é sinônimo de qualidade. Uma API pequena, completa, coerente e bem documentada pode obter a pontuação máxima.
- Deploy em nuvem é opcional, salvo orientação posterior do professor.
- Credenciais, senhas, tokens e chaves de APIs não deverão ser expostos no repositório. Quando necessário, deverão ser utilizadas variáveis de ambiente e arquivos de exemplo.
- O professor deverá conseguir acessar o repositório e reproduzir a aplicação com base nas instruções fornecidas.
- O trabalho deverá ser autoral. Bibliotecas, frameworks e referências podem ser utilizados, mas o estudante deverá compreender e explicar as partes relevantes da solução.

EXEMPLOS DE TEMAS POSSÍVEIS

Os exemplos abaixo têm caráter exclusivamente inspirador. **O estudante não precisa escolher um deles:**

- API de tarefas e categorias.
- API de biblioteca e empréstimos.
- API de agenda e compromissos.
- API de catálogo de produtos.
- API de receitas e ingredientes.
- API de filmes/séries assistidos e avaliações.
- API de eventos e inscrições.
- API de viagens com integração opcional à REST Countries.
- API de coleção temática com integração opcional a serviço público externo.